Hyperspectral Image Denoising via Clustering-Based Latent Variable in Variational Bayesian Framework

Peyman Azimpour, Tahereh Bahraini, and Hadi Sadoghi Yazdi

***Abstract*— The hyperspectral-image (HSI) noise-reduction step is a very significant preprocessing phase of data-quality enhancement. It has been attracting immense research attention in the remote sensing and image processing domains. Many methods have been developed for HSI restoration, the goal of which is to remove noise from the whole HSI cube simultaneously without considering the spectral–spatial similarity. When a noise-removal algorithm is used globally to the entire data set, it would not eliminate all levels of noise, effectively. Furthermore, most of the existing methods remove independent and identically distributed (i.i.d.) Gaussian noise. The real scenarios are much more complicated than this assumption. The complexity created by natural noise that has a non-i.i.d. structure leads to inefficient methods containing underestimation and invalid performance. In this article, we calculated the spatial–spectral similarity criteria by defining a set of clustering-based latent variables (CLVs) in a Bayesian framework to improve the robustness. These criteria can be extracted using the clustering operators. Then, by applying the CLV to the variational Bayesian model, we investigated a new low-rank matrix factorization denoising approach based on the proposed clustering-based latent variable (CLV-LRMF) to remove noise with the non-i.i.d. mixture of Gaussian structures. Finally, we switched to the GPU for MATLAB implementation to reduce the runtime. The experimental results show that the performance has been improved by applying the proposed CLV and demonstrate the effectiveness of the proposed CLV-LRMF over other state-of-the-art methods.**

***Index Terms*— Clustering-based latent variable (CLV), GPU, hyperspectral image (HSI), image denoising, low-rank matrix factorization (LRMF), mixture noise, nonindependent and identically distributed (non-i.i.d.) noise modeling.**

I. Introduction

HYPERSPECTRAL images (HSIs) capturing over hundreds of electromagnetic spectra bands enable the spectral signature extraction for observing objects or materials [1]. Due to the limited resolution of the HSI sensors and the existing noise, the noise-suppression or noise-reduction process is a basic and important preprocessing step to improve the quality of the data sets for the next processing steps (e.g., classification, feature extraction, unmixing or endmember extraction, and target detection). HSI's noise level depends on the sensor, the distance between the sensor and the area of imaging, atmospheric conditions, and various other factors [2]. The noise usually originates from equipment limitations, such as dark current, calibration error, fluctuations in power supply, sensor sensitivity, photon effects, and nonuniformity of the detector response. Furthermore, due to the limited radiance energy of the received signal and, generally, the narrow bandwidth of the HS receiver, the energy captured by each sensor might be weak, which makes it more apparent the effect of all kinds of noise in the HSI [3]. Although the HSI denoising approaches based on the low-rank matrix factorization (LRMF) mode such as that in [3] and [4] have displayed their effectiveness in certain noise types, these methods used only the low-rank characteristics of the adjacent areas of the HSI images in their models. There is no algorithm to distinguish the rank of a specific given cluster-based area, i.e., all similar pixels of the HSI data, which have the same spectral–spatial characteristics, and fall into the same cluster and, further, procure unique decomposition. We proposed new HSI denoising methods to alleviate these problems by using the clustering-based latent variable (CLV) in the variational Bayesian (VB) inference and extracted it by using some suitable clustering method. This idea is generalized with a formulation in this framework. In addition, the best available methods approximate noise with the independent and identically distributed (i.i.d.) structure. Despite the most existing algorithms, in this work, we use a non-i.i.d. mixture of Gaussian (NMoG) structure to model the real noise. In addition, we adjust the noise probabilistic model to estimate its characteristics. The main contributions of the proposed method [that has been named the low-rank matrix factorization denoising approach based on the proposed clustering-based latent variable (CLV-LRMF)] are listed as follows.

1) We propose a novel mixture noise-removal method using the CLV technique to exploit the spatial–spectral similar structures and characteristics of the HSI data. The existing studies have not been presented to exploit the spatial–spectral information using a CLV idea for HSI restoration. We use this in a statistical Bayesian framework to exploit properly the spatial–spectral self-similarity structures in the denoising process to increase the robustness of it.

Manuscript received January 26, 2019; revised May 31, 2019; accepted April 20, 2020. Date of current version March 25, 2021. *(Corresponding author: Hadi Sadoghi Yazdi.)*

Peyman Azimpour is with the Department of Civil Engineering, Faculty of Engineering, Ferdowsi University of Mashhad, Mashhad 91779-48974, Iran (e-mail: peyman.azimpour@mail.um.ac.ir).

Tahereh Bahraini is with the Department of Electrical Engineering, Faculty of Engineering, Ferdowsi University of Mashhad, Mashhad 91779-48974, Iran (e-mail: bahraini.tahereh@mail.um.ac.ir).

Hadi Sadoghi Yazdi is with the Department of Computer, Faculty of Engineering, Ferdowsi University of Mashhad, Mashhad 91779-48974, Iran (e-mail: h-sadoghi@um.ac.ir).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TGRS.2019.2939512.

Digital Object Identifier 10.1109/TGRS.2019.2939512

0196-2892 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:06:47 UTC from IEEE Xplore. Restrictions apply.

2) We build a measurement clustering model, which is the assemblage of the relations between every CLV and indicator variables that determines which pixel belongs to the CLV. In this way, we use several clustering methods to extract the CLV map such as zoning, *k*-means [5], robust k-means (Rk-means), fuzzy c-means (FCM) [6], and robust FCM (RFCM). By adding a 3-D Gaussian filter, we design and use the Rk-means and RFCM methods that are the resistant version of k-means and FCM.
3) We carry out the CLV step with GPU processing to reduce the runtime.

A. Related Work

Over the past decades, many researchers have been encouraged to improve the HSI's quality, which has led to the presentation of various denoising methods with different frameworks, including principal component analysis (PCA) [7], tensor analysis [8], wavelet filtering [9], [10], deep-learning-based methods [12], low-rank representation (LRR) [3], sparse representation (SR) [14], total variation (TV) [15], [16], and so on. Among these methods, the conventional 1-D and 2-D denoising methods are the simplest ways to reduce the noise level. These work in the form of pixel by pixel [2] or band by band, e.g., the block-matching 3-D (BM3D) filtering [18], K-singular value decomposition (K-SVD) [19], and other works in [20], [21]. They often occur in the low-quality results, because they ignore the spectral signatures or the neighboring pixel correlations.

Some of the recent methods focus on promoting quality by employing the spatial and spectral information correlations, and among them, the most popular is PCA [7]. In the following performance enhancement of the denoising process, the spatial and spectral dependences have been studied and a multiple-spectral-band conditional random field (MSB-CRF) model has been proposed in a unified probabilistic framework in [22]. By assuming that the HSI behaves as multidimensional data, the methods based on multidimensional analysis have been proposed for noise reduction [23], [24]. Specifically, by assuming that the HSI data behave as a 3-D cube, many other state-of-the-art methods can be employed in the HSI noise-reduction process such as BM3D, video denoising-based BM3D (VBM3D) [25], and BM4D [26]. Recently, another group of methods based on multilinear structures—called the tensor-based image-recovery methods—has a successful performance [8], [27], [28]. As an example, Xue *et al.* [8] proposed a nonlocal low-rank regularized tensor-decomposition method using nonlocal similar patches within the HSI images. To improve the hyperspectral data quality, some methods use the wavelet transform such as the PCA method in [7], multiway Wiener filtering [9], or first-order roughness penalty (FORP) [10]. The partial differential equation (PDE)-based methods have been widely applied for HSI denoising.

Moreover, the deep-learning theory provides a prominent strategy to help with the denoising tasks. Some works of literature such as [11] and [12] used convolutional neural networks (CNNs) to extract the features of images and achieved suitable performance on denoising of a natural image. In [13], a deep-learning-based method is proposed to reduce the noise of the HSI using a nonlinear end-to-end mapping between the noisy and clean HSIs and the combined spatial–spectral CNN.

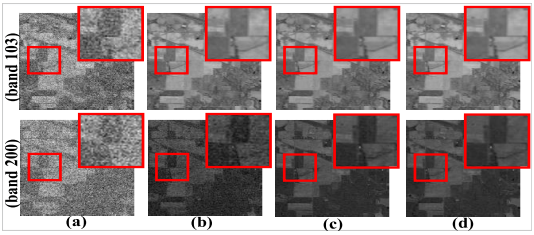


Fig. 1. Bands 103 and 200 of the Indian Pines data set. (a) Original noisy image. (b) MoG-RPCA [34]. (c) RegL1-ALM [35]. (d) LRTV [4].

Based on the low-rank characteristic of the HSI, the low-rank matrix approximation (LRMA) method has been implemented to HSI denoising in some literature such as [7], [29]–[33]. In our paper, the proposed CLV-LRMF method was compared with several low-rank matrix-based methods such as the NMoG-LRMF method [3], low-rank matrix-recovery (LRMR) method [7], (TV)-regularized low-rank matrix factorization (LRTV) method [4], robust PCA method to remove mixture of Gaussians noise (MoG-RPCA) [34], L1-regularized augmented Lagrange multiplier method (RegL1-ALM) [35], center weighted median (CWM) filter-based denoising method [36], TV-regularized low-rank tensor-decomposition method (LRTDTV) [15], and spatial–spectral total-variation-regularized local low-rank matrix-recovery (LLRSSTV) method [16]. In addition, we compared the obtained results with the fast superpixel-based subspace low-rank learning (FSSLRL) method [17] as one of the clustering-based low-rank denoising methods, which assumes the HSI to be low rank in the spectral domain and uses the nuclear norm within the superpixel-based regions. Exploiting the spatial–spectral information has been widely considered in several problems such as HSI classification [37]–[39] and band selection [40], [41]. In [42], a cluster sparsity field (CSF) as a prior for HSI reconstruction is proposed, which models the inherent correlation between the measurements within each spectrum for a particular pixel and the similarity between the pixels. In addition, in [14], the authors used the local and global redundancy and correlation (RAC) in the spatial and spectral domains jointly. They used sparse coding to model the global RAC and local RAC in the spatial and spectral domains, respectively.

Modeling the HSI cube as a vector or matrix is not proper to use the more accurate spatial–spectral self-similarity structures and leads to suboptimal results. As shown in Fig. 1(a), bands 103 and 200 of Indian Pines are polluted by heavy noise. The state-of-the-art low-rank matrix methods (e.g., MoG-RPCA in Fig. 1(b) [34], RegL1-ALM in Fig. 1(c) [35], and LRTV Fig. 1(d) [4]) could not provide accepted restoration results in all bands, and still, some parts of the noise remain. For instance, in Fig. 1(b), the MoG-RPCA has reached a fairly clear image in band 103, but in band 200, the stripes are still visible.

B. Organization of This Article

The rest of this article is formed as follows. The proposed model and the corresponding variational inference algorithm

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:06:47 UTC from IEEE Xplore. Restrictions apply.

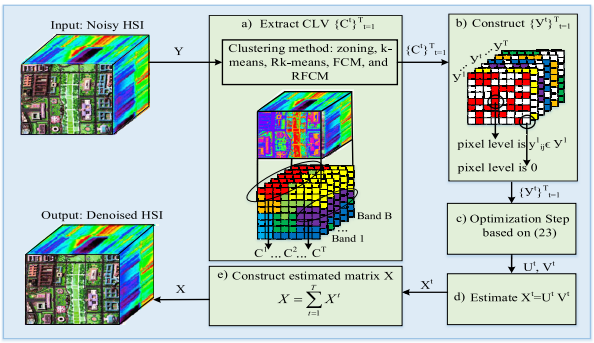


Fig. 2. Diagram of the proposed method.

are presented in Section II. The experimental results are shown in Section III. Finally, the conclusions are illustrated in Section IV.

II. Proposed Method

A. Motivation

In this section, the formulation of HSI data destruction is presented. Let we show a clean HSI data set with a matrix $X \in \mathbb{R}^{MN\times B}$, where MN and B are the spatial ($M \times N$ for an image with $M \times N$ pixels) and spectral dimensions of the HSI data set, respectively, and Y is the observed noisy HSI data that are contaminated by some types of noises, which are denoted by G and S as the Gaussian and sparse noises, respectively, where $Y \in \mathbb{R}^{MN\times B}$, $G \in \mathbb{R}^{MN\times B}$, and $S \in \mathbb{R}^{MN\times B}$. The goal is to estimate the clean HSI matrix X from Y. By assuming that noise is additive, we have

$$Y = X + G + S. \tag{1}$$

In our method, we assume the following LRMF model:

$$Y = UV + G + S. \tag{2}$$

where UV is the low-rank matrix factorization of X that $U \in \mathbb{R}^{MN\times p}$ is the endmember matrix or a random dictionary, and $V \in \mathbb{R}^{p\times B}$ is the abundance or random coefficient matrix. Assume that the rank p of the matrix UV is fixed.

Fig.2 depicts the flow of the process involved in our proposed CLV-LRMF method and its implementation in dealing with noisy HSI cubes. All bands are selected from the noisy input HSI Y. The selected bands of the HSI data as matrix Y are processed using the block [Fig. 2(a)] to extract CLV $\{C^t\}_{t=1}^T$ in the spatial and spectral domains. In this block, the clustering methods are exploited to group the similar region as the same class. We invoked the CLV as the spatial–spectral self-similarity structures to explain and cluster the similar rich structures across the whole data cube. The extracted similar structures in the same cluster $\{C^t\}_{t=1}^T$ that is the output of this block is given to the next block [Fig. 2(b)]. In this block, each class C^t is changed to make the related matrix $\mathcal{Y}^t$. So that if the pixel y_{ij} (located in ith row and jth column of Y) is a member of C^t, the element y_{ij}^t (located in the ith row and jth column of $\mathcal{Y}^t$) is equal to y_{ij}, or else, it is 0. $\{\mathcal{Y}^t\}_{t=1}^T$ is the output of this. In the next step, the optimization problem that is defined in (23) is applied to $\{\mathcal{Y}^t\}_{t=1}^T$ in the VB framework to obtain the optimal values $\mathcal{U}^t$ and $\mathcal{V}^t$. In this step, the noise of each band is modeled with a different mixture of Gaussian distributions, and their parameters are defined and approximated using the prior distributions. After forming the matrix $\mathcal{X}^t$ in block [Fig. 2(d)], finally, X is constructed in Fig. 2(e).

B. Clustering-Based-Latent-Variable (CLV) Step

As shown in Fig. 2, in our proposed method, we extracted the clustering-based latent variable (CLV) C_t in the CLV step. Therefore, we have the following HSI degradation model for the tth cluster C_t:

$$\mathcal{Y}^t = \mathcal{X}^t + \mathcal{G}^t + \mathcal{S}^t \tag{3}$$

where $\mathcal{X}^t$ as a part of X includes every pixel of X that is dependent on C_t. $\mathcal{Y}^t \in \mathbb{R}^{MN\times B}$ is an extracted matrix that contains all the pixels of Y, which are the members of C_t, and for other pixels, the element of $\mathcal{Y}^t$ is zero. The element $\mathcal{Y}_{ij}^t$ that is located in the ith row and jth column of $\mathcal{Y}^t$ is a member of C_t with a membership probability function $q(C_{ij}^t)$. In the ideal case, where the pixel $y_{ij} \in Y$ contains only the tth cluster C^t (which is denoted by $\mathcal{Y}_{ij}^t$ and assigned to matrix $\mathcal{Y}^t$), the membership probability function is obtained as

$$q\left(C_{ij}^t\right) = \begin{cases} 1, & y_{ij} \in C^t \\ 0, & y_{ij} \notin C^t. \end{cases} \tag{4}$$

In the fuzzy-CLV that is extracted using a fuzzy clustering method, the pixel $y_{ij} \in Y$ may be a member of more than one cluster, and therefore, in this case

$$\sum_{t=1}^{T} q\left(C_{ij}^t\right) = 1. \tag{5}$$

We use the binary mode of it that is represented in (4). According to the proposed method, first, we estimate all $\{\mathcal{X}^t\}_T^{t=1}$ and then reconstruct X as

$$X = \sum_{t=1}^{T} \mathcal{X}^t. \tag{6}$$

For $\mathcal{Y}^t$ (as the observed HSI data related to the tth cluster C_t), we have the following LRMF model:

$$\mathcal{Y}^t = \mathcal{U}^t\mathcal{V}^t + \mathcal{G}^t + \mathcal{S}^t. \tag{7}$$

$\mathcal{Y}^t$ is destroyed by $\mathcal{G}^t \in \mathbb{R}^{MN\times B}$ and $\mathcal{S}^t \in \mathbb{R}^{MN\times B}$ as the Gaussian and sparse noises (related to the tth cluster C_t) and factorized to $\mathcal{U}^t \in \mathbb{R}^{MN\times p}$ and $\mathcal{V}^t \in \mathbb{R}^{p\times B}$ matrices. We assume the ith columns of $\mathcal{U}^t$ and $\mathcal{V}^t$ to be $\mathfrak{u}_i^t$ and $\mathfrak{v}_i^t$, which are constructed from the Gaussian distributions with the following q as their density functions:

$$q\left(\mathfrak{u}_i^t\right) = N\left(\mathfrak{u}_i^t|\mu_{\mathfrak{u}_i^t}, \Sigma_{\mathfrak{u}_i^t}\right) \tag{8}$$

$$q\left(\mathfrak{v}_i^t\right) = N\left(\mathfrak{v}_i^t|\mu_{\mathfrak{v}_i^t}, \Sigma_{\mathfrak{v}_i^t}\right) \tag{9}$$

where $\mu_{\mathfrak{u}_i^t}$ (or $\mu_{\mathfrak{v}_i^t}$) and $\Sigma_{\mathfrak{u}_i^t}$ ($\Sigma_{\mathfrak{v}_i^t}$) are the mean and variance of the Gaussian distribution $q(\mathfrak{u}_i^t)$ [$q(\mathfrak{v}_i^t)$]. To create these, we have

$$\mu_{\mathfrak{u}_i^t} = \left\{\sum_{j,k} \bar{\mathcal{Z}}^t \bar{\mathfrak{h}}^{g,t}\left(\mathcal{Y}_{ij}^t - \bar{\mathfrak{w}}^{g,t}\right)\bar{\mathfrak{v}}^t\right\}\Sigma_{\mathfrak{u}_i^t} \tag{10}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:06:47 UTC from IEEE Xplore. Restrictions apply.

$$\mu_{\mathfrak{v}_i^t} = \left\{ \sum_{j,k} \bar{\mathcal{Z}}^t \bar{\mathfrak{h}}^{g,t} \left(\mathcal{Y}_{ij}^t - \bar{\mathfrak{w}}^{g,t} \right) \bar{\mathfrak{u}}^t \right\} \Sigma_{\mathfrak{v}_i^t} \tag{11}$$

where $(\bar{\cdot})$ denotes the expectation values of the variables; $\bar{\mathcal{Z}}^t$, $\bar{\mathfrak{h}}^{g,t}$, $\bar{\mathfrak{w}}^{g,t}$, and $\bar{\mathfrak{v}}^t$ are the mean of $\mathcal{Z}_{ijk}^t$, $\mathfrak{h}_{jk}^{g,t}$, $\mathfrak{w}_{jk}^{g,t}$, and $\mathfrak{v}_j^t$, respectively; and k is the number of Gaussian components in the mixture of Gaussian noise that is defined as follows. $\Sigma_{\mathfrak{u}_i^t}$ and $\Sigma_{\mathfrak{v}_i^t}$ can be obtained from

$$\Sigma_{\mathfrak{u}_i^t} = \left\{ \sum_{j,k} \bar{\mathcal{Z}}^t \bar{\mathfrak{h}}^{g,t} \overline{\| \mathfrak{v}_i^t \|^2} + \bar{\Gamma}^t \right\}^{-1} \tag{12}$$

$$\Sigma_{\mathfrak{v}_i^t} = \left\{ \sum_{j,k} \bar{\mathcal{Z}}^t \bar{\mathfrak{h}}^{g,t} \overline{\| \mathfrak{u}_i^t \|^2} + \bar{\Gamma}^t \right\}^{-1} \tag{13}$$

where $\| \cdot \|^2$ denotes norm 2, $\Gamma^t = \mathrm{diag}(\overline{\gamma}^t)$, and the density function of γ^t is

$$q\left(\gamma^t\right) = \mathrm{Gam}\left(\gamma^t | \xi^t, \delta^t\right) \tag{14}$$

where ξ^t and δ^t as the hyperparameter update in each iteration. In addition, in (10)–(13), $\bar{\mathcal{Z}}_{ijk}^t$ is the mean of $\mathcal{Z}_{ijk}^t$, which is defined as

$$\mathcal{Z}_{ij}^t \sim \mathrm{Multinomial}\left(\mathcal{Z}_{ij}^t | \pi_j^t\right) \tag{15}$$

where $\mathcal{Z}_{ij}^t$ has the multinomial$(\cdot)$ distribution and update equation

$$q\left(\mathcal{Z}_{ij}^t\right) = \prod_{k=1}^{K} \pi_{ijk}^{\mathcal{Z}_{ij}^t} \tag{16}$$

where π_j^t can written as

$$\pi_j^t \sim \mathrm{Dir}\left(\pi_j^t | \alpha_0\right) \tag{17}$$

where Dir$(\cdot)$ represents the Dirichlet distribution and has the following update equation:

$$q\left(\pi_j^t\right) = \prod_{k=1}^{K} {\pi_{jk}^t}^{\alpha_{jk}^t - 1} \tag{18}$$

where α_{jk}^t is the hyperparameter updated in each iteration. In (10)–(13), $\mathfrak{w}_{jk}^{g,t}$ and $\mathfrak{h}_{jk}^{g,t}$ are defined as the mean and variance of Gaussian noise $\mathcal{G}^t$. Let $\mathcal{G}^t$ has the mixture of Gaussian distribution and can be represented as a linear superposition of Gaussian. The element located in the ith row and jth column of $\mathcal{G}^t$ can be written as g_{ij}^t with the following distribution:

$$g_{ij}^t \sim \prod_{k=1}^{K} N\left(g_{ij}^t | \mathfrak{w}_{jk}^{g,t}, \frac{1}{\mathfrak{h}_{jk}^{g,t}} \right)^{\mathcal{Z}_{ijk}^t}. \tag{19}$$

In (19), $\mathfrak{w}_{jk}^{g,t}$ and $\mathfrak{h}_{jk}^{g,t}$ are generated from a two-level distribution

$$q\left(\mathfrak{w}_{jk}^{g,t}, \mathfrak{h}_{jk}^{g,t}\right) = \prod_{k=1}^{K} N\left(\mathfrak{w}_{jk}^{g,t} | m_{jk}^t, \frac{1}{\beta_{jk}^t, \mathfrak{h}_{jk}^{g,t}} \right) \mathrm{Gam}\left(\mathfrak{h}_{jk}^{g,t} | c_{jk}^t, d_{jk}^t\right) \tag{20}$$

where in (20), Gam(.) represents the Gamma distribution and m_{jk}^t, β_{jk}^t, c_{jk}^t, and d_{jk}^t as the hyperparameters are calculated from data. The other variable in (3) is $\mathcal{S}^t$ to represent the sparse noise that corrupts the pixels of $\mathcal{Y}^t$. $\mathfrak{s}_{ij}^t$ as the element located in the ith row and jth column of the matrix $\mathcal{S}^t$ has Laplace distribution and can be formulated as

$$\mathfrak{s}_{ij}^t \sim \mathrm{Lap}\left(\mathfrak{s}_{ij}^t | \mathfrak{w}_{jk}^{\mathfrak{s},t}, \mathfrak{h}_{jk}^{\mathfrak{s},t}\right) \tag{21}$$

where Lap$(\cdot)$ denotes Laplace distribution. The two-level distribution for $\mathfrak{w}_{jk}^{\mathfrak{s},t}$ and $\mathfrak{h}_{jk}^{\mathfrak{s},t}$ is obtained as

$$q\left(\mathfrak{w}_{jk}^{\mathfrak{s},t}, \mathfrak{h}_{jk}^{\mathfrak{s},t}\right) = \prod_{k=1}^{K} N\left(\mathfrak{w}_{jk}^{\mathfrak{s},t} | m_{jk}^t, \frac{1}{\beta_{jk}^t, \mathfrak{h}_{jk}^{\mathfrak{s},t}} \right) Wish\left(\mathfrak{h}_{jk}^{\mathfrak{s},t} | c_{jk}^t, d_{jk}^t\right) \tag{22}$$

where Wish$(\cdot)$ denotes the Wishart distribution.

C. HSI-Estimation Procedure

In our model, the maximum *a posteriori* (MAP) estimator theory has been used to find optimal $\mathcal{U}^t$ and $\mathcal{V}^t$ for the purpose of reconstructing the clean HSI data $\mathcal{X}^t = \mathcal{U}^t \mathcal{V}^t$. It needs to use the prior probability density functions of the variables, latent variables, and hyperparameters. Therefore, using an MAP estimator, we maximize the following posterior probability $\mathcal{P}$ of all involved variables:

$$\arg\max_{\mathcal{U}^t, \mathcal{V}^t} \mathcal{P}\left(\mathcal{U}^t, \mathcal{V}^t, \mathcal{Z}^t, C^t, \mathfrak{w}^{g,t}, \mathfrak{h}^{g,t}, \mathfrak{w}^{s,t}, \mathfrak{s}^{g,t}, \pi^t, \gamma^t | \mathcal{Y}^t\right) \tag{23}$$

where we define $\mathcal{Z}^t = (\mathcal{Z}_{ij}^t)_{MN \times B}$, $\mathfrak{w}^{g,t} = (\mathfrak{w}_{jk}^{g,t})_{B \times K}$, $\mathfrak{h}^{g,t} = (\mathfrak{h}_{jk}^{g,t})_{B \times K}$, $\mathfrak{w}^{S,t} = (\mathfrak{w}_{jk}^{S,t})_{B \times K}$, $\mathfrak{h}^{S,t} = (\mathfrak{h}_{jk}^{S,t})_{B \times K}$, $\pi^t = (\pi_1, ..., \pi_B)$, $\gamma^t = (\gamma_1, ..., \gamma_R)$, and $C = (C^1, \ldots, C^t, \ldots, C^T)$. $t \in [1, T]$ and T are the integer and the number of proposed CLVs (that can be adaptively determined for every HSI data set), respectively. To obtain posterior probability in (23), we used the VB method. In the variational inference concept, the posterior distribution $\mathcal{P}(\mathcal{U}^t, \mathcal{V}^t, \mathcal{Z}^t, C^t, \mathfrak{w}^{g,t}, \mathfrak{h}^{g,t}, \mathfrak{w}^{s,t}, \mathfrak{s}^{g,t}, \pi^t, \gamma^t | \mathcal{Y}^t)$ is approximated using a variational distribution $p(\mathcal{U}^t, \mathcal{V}^t, \mathcal{Z}^t, C^t, \mathfrak{w}^{g,t}, \mathfrak{h}^{g,t}, \mathfrak{w}^{s,t}, \mathfrak{s}^{g,t}, \pi^t, \gamma^t)$, where the distribution p is composed of a family of distributions to approximate the distribution p to the main posterior distribution $\mathcal{P}$. Among these, there is an optimal and most appropriate distribution q to maximize this similarity. The dissimilarity function $d(p; \mathcal{P})$ measures the lack of resemblance and inconsistency between $\mathcal{P}$ and p [43]. We employed the Kullback–Leibler (KL) divergence as the most common kind of variational Bayes to measure the dissimilarity function $d(p; \mathcal{P})$. The reverse KL-divergence is

$$d_{KL}(p||\mathcal{P}) = \sum_{\theta} p\left(\mathcal{U}^t, \mathcal{V}^t, \mathcal{Z}^t, C^t, \mathfrak{w}^{g,t}, \mathfrak{h}^{g,t}, \mathfrak{w}^{s,t}, \mathfrak{s}^{g,t}, \pi^t, \gamma^t\right) \times \log \frac{p\left(\mathcal{U}^t, \mathcal{V}^t, \mathcal{Z}^t, C^t, \mathfrak{w}^{g,t}, \mathfrak{h}^{g,t}, \mathfrak{w}^{s,t}, \mathfrak{s}^{g,t}, \pi^t, \gamma^t\right)}{\mathcal{P}(\mathcal{U}^t, \mathcal{V}^t, \mathcal{Z}^t, C^t, \mathfrak{w}^{g,t}, \mathfrak{h}^{g,t}, \mathfrak{w}^{s,t}, \mathfrak{s}^{g,t}, \pi^t, \gamma^t | \mathcal{Y}^t)} \tag{24}$$

where $\mathcal{P}$ is still the theoretic distribution. We try to match p as approximation and minimize $d_{\mathrm{KL}}(p||\mathcal{P})$ as the dissimilarity

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:06:47 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 CLV-LRMF Algorithm

Input: Noisy HSI data set $Y \in \mathbb{R}^{MN\times B}$, the number of mixture components for Gaussian noise is K.
Output: $\mathcal{U}_{opt}^t = \mathcal{U}^t$, and $\mathcal{V}_{opt}^t = \mathcal{V}^t$.
Initialization: Set initial values to all hyper-parameters and low rank components $\mathcal{U}_0^t$, $\mathcal{V}_0^t$, and $I = 1$.
1: Extract all CLVs C^t and $q(C_{ij}^t)$ in CLV step.
2: for $t = 1 : T$
3: While not converged do:
4: Update approximate posterior probability noise components $\mathcal{Z}_I^t$ and π_I^t by equations (16)-(18).
5: Update approximate posterior probability noise components $\mathfrak{h}_I^{t,g}$, $\mathfrak{w}_I^{t,g}$, $\mathfrak{h}_I^{t,s}$ and $\mathfrak{w}_I^{t,s}$ by equations (20)-(22).
6: Update approximate posterior probability of low rank components $\mathcal{U}_I^t$ and $\mathcal{V}_I^t$ by equations (8)-(9).
7: Update approximate posterior probability of for noise parameters γ_I^t by equation (14).
8: go to next Iteration so $I = I + 1$
9: end while
10: Construct $\mathcal{X}^t = \mathcal{U}^t\mathcal{V}^t$.
11: end for
12: Construct X by equation (6) .

between p and $\mathcal{P}$. Suppose that the approximation of posterior distribution $p(\mathcal{U}^t, \mathcal{V}^t, \mathcal{Z}^t, C^t, \mathfrak{w}^{g,t}, \mathfrak{h}^{g,t}, \mathfrak{w}^{s,t}, \mathfrak{s}^{g,t}, \pi^t, \gamma^t)$ possesses a factorized form as follows:

$$\begin{aligned} &p\left(\mathcal{U}^t, \mathcal{V}^t, \mathcal{Z}^t, C^t, \mathfrak{w}^{g,t}, \mathfrak{h}^{g,t}, \mathfrak{w}^{s,t}, \mathfrak{s}^{g,t}, \pi^t, \gamma^t\right) \\ &\quad = \prod_i q\left(\mathfrak{u}_i^t\right) q\left(\mathfrak{v}_i^t\right) \prod_{ij} q\left(\mathcal{Z}_{ij}^t\right) q\left(C_{ij}^t\right) \\ &\qquad \times \prod_j q\left(\mathfrak{w}_j^{g,t}, \mathfrak{h}_j^{g,t}\right) q\left(\mathfrak{w}_j^{s,t}, \mathfrak{h}_j^{s,t}\right) q\left(\pi_j^t\right) \prod_l q\left(\gamma_l^t\right) \end{aligned} \tag{25}$$

where the update equation of each component is previously presented. Then, the proposed CLV-LRMF method can be summarized as shown in Algorithm 1.

III. Experimental Results

We conducted a lot of experiments in several scenarios with different conditions to evaluate and analyze the performance of the proposed CLV-LRMF method in comparison with several other noise-reduction methods, which can be listed as follows.

1) We compared the CLV-LRMF method with nine denoising algorithms (i.e., NMoG-LRMF [3], LRMR [7], LRTV [4], MoG RPCA [34], RegL1-ALM [35], CWM [36], FSSLRL [17], LRTDTV [15], and LLRSTV [16]).
2) Simulations were performed with two types of HSI data: synthetic (i.e., Washington DCMall and RemoteImage) and real data sets (i.e., Urban and Indian Pines).
3) The effect of six types of noises was investigated in the synthetic data.
4) Six evaluation criteria were used to compare the performance of methods in the synthetic data sets (i.e., PSNR, SSIM, FSIM, ERGAS, RMSE, and Time).
5) Three approaches were used and defined to compare the performance of methods in the real data sets (i.e., horizontal and vertical profile of bands and the no-reference quality assessment method [44]).
6) Finally, we used the GPU processors to reduce the runtime of the CLV-LRMF method compared with other methods.

All the experiments were conducted on a computer with a 4-GHz CPU and 32-GB RAM. The proposed algorithm and other methods were carried out under MATLAB R2016a.

A. HSI Data Set

The synthetic and real HSI data sets were employed to consider the effectiveness and stability of the proposed method.

1) Synthetic HSI Experiments: Washington DCMall with the size of $1208 \times 307 \times 191$ and RemoteImage [45] with the size of $205 \times 246 \times 96$ are used as the synthetic data and the simulated noise is added to them. In order to prepare the data, some simple preprocessing steps are done on the data cube, such as cropping the main part of the HSI, resizing (Washington DCMall to $200 \times 200 \times 160$ and RemoteImage to $200 \times 200 \times 89$), and normalizing the gray-level values of pixels into [0, 1].

2) Real HSI Experiments: Two real data sets were used in the experiments. The Urban with the size of $307 \times 307 \times 210$ is collected by the hyperspectral digital imagery collection experiment (HYDICE) sensors. The second data set is Indian Pines that was captured by the NASA AVIRIS sensor in 1992 over the Indian Pines test site with a size of $145 \times 145 \times 220$. Several bands of real data are often polluted by heavy noise originating from the water and atmosphere [46]. Removing defective bands is the usual step, but we used all the data without removing any bands to more rationally verify the robustness of the CLV-LRMF method in heavy-noise scenarios.

B. Effect of Noise Types

We used six kinds of noises in the simulations to generate synthetic noisy HSI data sets, which are described in the following.

1) I.I.D. Gaussian Noise: In the data cube, all bands are corrupted by the zero-mean i.i.d. Gaussian noise $N(0, \sigma^2)$ with $\sigma = 0.05$. In these bands, the noise levels are the same.

2) Non-i.i.d. Gaussian Noise: We added the Gaussian noise with various powers σ_n to the clean HSI for simulating the synthetic data. The noise levels are different in all bands. The signal-to-noise-ratio (SNR) value of each band is formed from a uniform distribution with a value in the interval [10, 20] dB.

3) Non-i.i.d. Gaussian and Stripe Noise: The non-i.i.d. Gaussian noise is added to all bands, and the stripe is randomly simulated for 40 bands in the Washington DCMall and RemoteImage data. The number of stripes is from 20 to 40 lines.

4) Non-i.i.d. Gaussian & Deadline Noise: Each band is destroyed by the non-i.i.d. Gaussian noise. After that, the deadlines are added to 40 bands that are selected randomly. The number of deadlines in each band changes randomly between 5 and 15, and the width of this is 1 pixel.

5) Non-i.i.d. Gaussian and Impulse Noise: The non-i.i.d. Gaussian noise and the impulse noise are added to all bands and to 40 randomly selected bands, respectively. The percentage of impulse (intensity) is from 50% to 70%.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:06:47 UTC from IEEE Xplore. Restrictions apply.

6) Mixture Noise: It is assumed that the data were destroyed by all kinds of noises mentioned in the previous cases.

C. Evaluation Criteria for Synthetic Data Sets

To evaluate quantitatively the effectiveness of our method and other competing methods, we have applied six evaluation criteria and mean values on all bands in the simulations of the synthetic data sets, e.g., PSNR, SSIM, FSIM, ERGAS, RMSE, and Time [16], [47], [48]. Time shows computational cost. To reduce the runtime of the LV-LRMF method, we carried out the LV step with a GPU in Section IV. The larger values of MPSNR, MSSIM, and MFSIM (mean values of PSNR, SSIM, and FSIM) and the smaller values of MERGAS, MRMSE (mean of ERGAS and RMSE), and TIME indicated the better performance of a method to restore the images.

D. Evaluation Criteria for Real Data Sets

1) Horizontal Profile: The horizontal mean profile of a band refers to the mean of each row of it. The horizontal and vertical axes of the output curve represent the number of rows and the mean of the digital number (DN) values of each of them, respectively.

2) Vertical Profile: We acquired this by computing the mean of the DNs located in each column of a band. In this curve, the vertical axis represents the mean of the DN values of each column and the horizontal axis shows the column number.

3) Quality-Assessment Method [44]: We used no-reference HSI quality assessment via quality-sensitive feature learning [44] to evaluate the denoising performance of the real data.

E. Finding CLV (C_t)

We defined and extracted the CLV, $\{C_t|t = 1, \ldots, T\}$, and then applied the proposed CLV-LRMF denoising algorithm to the new domain CLV instead of the HSI data cube. Our purpose for using the CLV is based on that the natural images often have a spatially local and nonlocal similarity. In addition, in the HSIs, these similarities also are repeated in the spectral domain, since the homogeneous materials have similar spectral signatures. We assume that a noisy HSI can be displayed using the spectral–spatial homogeneous latent variables according to the spectral–spatial similarity. Thus, each CLV contains similar spectra from the local and nonlocal similar pixels and this provides a strong prior for the improved denoising process. We suggested a methodology based on the clustering algorithms as an operator to exploit the CLV. Choosing these operators can be influenced by different factors, such as the HSI data set, selective noise removal method, hardware, and time limitations, and other conditions related to the issue. Therefore, we used and evaluated several clustering techniques as the extraction operator, e.g., zoning, k-means [5], Rk-means, FCM [6], and RFCM.

To design RFCM and Rk-means, we employed a 3-D Gaussian filtering scheme for extracting robust CLV and rejecting the effect of noises. According to Fig. 3, these operators have been used to extract the CLV in the CLV-LRMF method in the Washington DCMall data set. As shown in Fig. 4 and Table I, all results are very close, and RFCM has a worse performance with 0.3-dB difference compared with zoning as

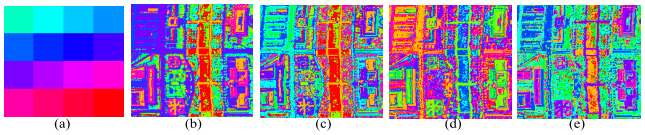


Fig. 3. CLV extraction map of (a) zoning, (b) k-means, (c) Rk-means, (d) FCM, and (e) RFCM.

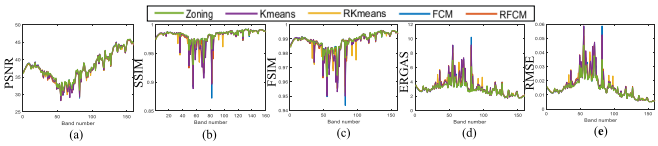


Fig. 4. CLV-LRMF results using zoning, k-means, Rk-means, FCM, and RFCM extraction operators for mixture noise and the criteria (a) PSNR, (b) SSIM, (c) FSIM, (d) ERGAS, and (e) RMSE.

TABLE I

PERFORMANCE EVALUATION RESULTS FOR CLV-LRMF METHOD USING DIFFERENT CLV EXTRACTION OPERATORS, SUCH AS ZONING, K-MEANS, RK-MEANS, FCM, AND RFCM

Criteria	Noisy HSI	Without Clustering	Zoning	k-means [5]	Rk-means	FCM [6]	RFCM
Mixture noise							
MPSNR	22.87210	36.47346	**38.08340**	37.85560	37.78090	37.84960	37.76690
MSSIM	0.65310	0.97418	**0.98246**	0.97940	0.97917	0.97967	0.97937
MFSIM	0.83840	0.98381	**0.98898**	0.98760	0.98745	0.98784	0.98773
MERGAS	17.59190	3.62152	**2.99180**	3.20140	3.22170	3.17410	3.19420
MRMSE	0.08380	0.01898	**0.01449**	0.01573	0.01580	0.01555	0.01562
TIME (sec)	-	128.654	**130.214**	169.761	169.835	211.448	209.154

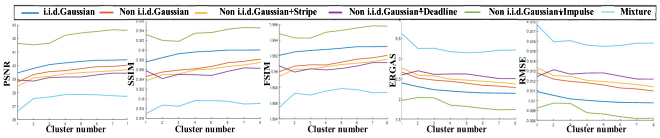


Fig. 5. Effect of cluster number in the performance of the proposed CLV-LRMF method on six kinds of noise for the synthetic DCMall data set for (a) PSNR, (b) SSIM, (c) FSIM, (d) ERGAS, and (e) RMSE.

the best operator. In the all next simulations, the RFCM was used for finding similar local and nonlocal areas (because of the robustness to all types of noises), wherein the similarities in the whole HSI cube were harnessed to form the C_t submatrix. Therefore, we used the structures with a high correlation, which are grouped together into the same C_t. It has pixels with similar noise information and minimal structural differences. We fixed the cluster number at 8 to balance the computational cost in the performance of the CLV-LRMF method. The initial rank has been assumed at 30 in the simulations, so the acceptable submatrix for each cluster should have at least 30 pixels. When the cluster number is increased, the number of pixels in each cluster decreases, which may cause a cluster with less than 30 samples. This limits the execution of the CLV-LRMF simulation code. In Fig. 5, we examined the effect of the cluster number in the performance of the proposed method and run the CLV-LRMF using RFCM with the cluster number varying from 1 to 8. The results of different cluster numbers on six kinds of noise over the Washington DCMall data set are illustrated in this. It can be easily observed that after the number is larger than 7, the performance of the method tends to be stable and not very sensitive to the choice of its value.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:06:47 UTC from IEEE Xplore. Restrictions apply.

TABLE II

PERFORMANCE COMPARISON OF ALL COMPETING METHODS ON THE WASHINGTON DCMALL AND REMOTEIMAGE DATA SETS

Data set	Washington DCMALL											RemoteImage									
Criteria	Noisy HSI	MoG RPCA	RegL1-ALM	LRMR	LRTV	CWM	FSSLRL	LRTDTV	LLRSTV	NMoG-LRMF	CLV-LRMF	Noisy HSI	MoG RPCA	RegL1-ALM	LRMR	LRTV	CWM	FSSLRL	LRTDTV	LLRSTV	NMoG-LRMF	CLV-LRMF
i.i.d Gaussian Noise																						
MPSNR	26.0204	39.5888	37.9120	39.2272	38.6906	38.0807	39.369	39.7771	40.1181	39.4652	**40.4567**	26.0212	37.2953	35.5797	37.6005	36.2866	35.7976	37.2175	37.3763	37.6872	37.3444	**37.7954**
MSSIM	0.76260	0.98799	0.98010	0.98666	0.98488	0.98079	0.98697	0.98840	0.98886	0.98762	**0.99319**	0.6954	0.97110	0.95151	0.96920	0.95873	0.95363	0.96815	0.96911	0.96906	0.97065	**0.97207**
MFSIM	0.89043	0.99230	0.98829	0.99091	0.99020	0.98869	0.99181	0.99235	0.9926	0.99205	**0.99319**	0.89077	0.98640	0.97961	0.98474	0.98132	0.98046	0.98566	0.98568	0.98531	0.98634	**0.98672**
MERGAS	11.4359	2.36350	2.87004	2.46158	2.61624	2.81471	2.42436	2.31489	2.21662	2.40023	**2.13371**	7.7569	2.1802	2.6431	2.0915	2.4353	2.5805	2.2002	2.15476	2.06711	2.1695	**2.0549**
MRMSE	0.05000	0.01072	0.01299	0.01117	0.01192	0.01274	0.011039	0.010547	0.010173	0.01090	**0.00973**	0.04999	0.01402	0.01698	0.01343	0.01565	0.01658	0.014151	0.013861	0.013282	0.01395	**0.01320**
Time (sec)	-	**13.579**	34.782	110.944	82.387	39.425	14.249	138.864	172.175	91.536	200.362	-	**7.939**	20.346	70.077	50.258	22.659	10.434	91.7669	101.274	74.167	105.387
Non i.i.d Gaussian Noise																						
MPSNR	24.9684	36.2165	36.3055	38.2014	36.8127	36.1950	37.3841	37.3581	37.808	38.8609	**39.5482**	20.6713	31.4354	30.3838	33.1171	32.2411	31.2262	32.5023	32.9234	33.4956	34.0062	**34.4967**
MSSIM	0.72704	0.96740	0.97095	0.98270	0.97772	0.97324	0.97876	0.98021	0.9806	0.98497	**0.98534**	0.43639	0.87845	0.84896	0.91681	0.90426	0.88262	0.90534	0.91343	0.92261	0.93703	**0.94414**
MFSIM	0.87439	0.98087	0.98253	0.98851	0.98528	0.98340	0.9867	0.98657	0.98738	0.99021	**0.99049**	0.77224	0.94928	0.94115	0.96344	0.95495	0.94973	0.96098	0.96139	0.96351	0.97211	**0.97459**
MERGAS	13.3154	3.76638	3.54668	2.78260	3.34398	3.58671	3.09538	3.11138	2.98846	2.63450	**2.48875**	15.1376	4.60892	5.00490	3.51840	4.12419	4.63294	3.88793	3.68300	3.50793	3.23069	**2.99916**
MRMSE	0.06301	0.01811	0.01695	0.01319	0.01583	0.01689	0.01463	0.01467	0.014234	0.01260	**0.01208**	0.09801	0.02991	0.03239	0.02272	0.02666	0.03000	0.02498	0.023803	0.022697	0.02087	**0.01935**
Time (sec)	-	**17.155**	34.962	111.050	82.777	39.452	29.478	166.9282	160.2858	96.839	169.942	-	28.440	**20.178**	67.998	47.132	22.402	23.132	111.497	100.802	72.328	93.854
Gaussian + Stripe Noise																						
MPSNR	24.3903	36.5415	36.2534	37.6670	36.6522	36.0409	37.2381	37.2687	37.6184	38.9785	**39.7031**	20.4069	30.8030	30.2434	32.3963	31.9915	30.9508	32.1432	32.4589	32.5141	33.8970	**34.1803**
MSSIM	0.70985	0.96879	0.97073	0.98126	0.97731	0.97206	0.97825	0.97979	0.98063	0.98570	**0.98765**	0.43037	0.88250	0.84663	0.91083	0.90161	0.87068	0.89905	0.90665	0.91295	0.93852	**0.94375**
MFSIM	0.86839	0.98232	0.98232	0.98747	0.98493	0.98271	0.98617	0.98627	0.98701	0.99061	**0.99171**	0.76581	0.95010	0.93968	0.95911	0.95317	0.94635	0.95808	0.95813	0.95815	0.97231	**0.97373**
MERGAS	14.2633	3.66531	3.55131	2.97806	3.39453	3.65299	3.14610	3.14454	3.04113	2.57277	**2.36640**	15.4787	4.90639	5.06092	3.84224	4.24040	4.75084	4.04753	3.86659	3.8365	3.22351	**3.08070**
MRMSE	0.06697	0.01791	0.01697	0.01410	0.01604	0.01724	0.01491	0.014894	0.014487	0.01226	**0.01131**	0.10019	0.03179	0.03273	0.02482	0.02741	0.03077	0.02605	0.024995	0.024766	0.02081	**0.01986**
Time (sec)	-	98.448	**34.718**	108.742	80.797	39.190	40.659	166.1483	161.7712	127.303	191.055	-	90.739	**20.468**	68.706	48.451	22.540	35.438	112.009	101.063	91.450	114.019
Gaussian + Deadline Noise																						
MPSNR	24.2383	34.6222	36.1432	37.5918	36.6609	35.9109	37.1291	37.167	36.8318	38.5419	**39.4384**	19.4011	24.0179	30.1319	31.3920	31.8500	30.6325	31.7452	31.9457	30.6994	33.5445	**33.8060**
MSSIM	0.70961	0.95778	0.97022	0.98043	0.97709	0.97135	0.97840	0.97967	0.97812	0.98411	**0.98618**	0.40405	0.86760	0.84660	0.89634	0.90309	0.87288	0.90347	0.90694	0.89697	0.93843	**0.94367**
MFSIM	0.86685	0.98154	0.98219	0.98725	0.98496	0.98246	0.98653	0.98638	0.9854	0.98977	**0.99117**	0.74590	0.94897	0.93940	0.95093	0.95276	0.94466	0.95659	0.95538	0.94513	0.97271	**0.97389**
MERGAS	14.5825	4.60364	3.61627	3.04316	3.41568	3.74124	3.19299	3.19729	3.47177	2.75869	**2.51073**	17.6724	10.4246	5.18352	4.56405	4.43064	5.07910	4.42224	4.32917	5.12837	3.38914	**3.29613**
MRMSE	0.06952	0.02190	0.01731	0.01456	0.01620	0.01770	0.01510	0.015113	0.016693	0.01323	**0.01215**	0.11451	0.06732	0.03355	0.02956	0.02864	0.02864	0.028584	0.027983	0.033263	0.03291	**0.02189**
Time (sec)	-	128.216	**35.136**	108.566	80.753	39.428	48.243	167.0639	159.6961	128.042	210.957	-	279.028	**20.377**	69.811	49.408	22.574	36.614	112.126	100.732	92.185	115.807
Gaussian + Impulse Noise																						
MPSNR	24.1917	36.4013	37.5233	39.8239	37.7336	37.3662	37.3718	38.4614	38.9737	41.8629	**42.6376**	19.1183	31.9491	32.9627	35.5532	33.8350	33.4891	33.2111	34.2844	35.1961	39.4008	**39.6589**
MSSIM	0.71481	0.97317	0.97843	0.98857	0.98273	0.98021	0.97962	0.98476	0.98534	0.99375	**0.99468**	0.37066	0.93813	0.89934	0.94693	0.93148	0.91746	0.91721	0.93367	0.94647	0.97775	**0.97949**
MFSIM	0.86791	0.98775	0.98677	0.99212	0.98850	0.98762	0.98718	0.98987	0.99031	0.99519	**0.99588**	0.73544	0.97292	0.95818	0.97530	0.96558	0.96417	0.96511	0.97006	0.97334	0.98746	**0.98727**
MERGAS	14.7015	3.69180	3.13170	2.31675	3.03526	3.16859	3.11308	2.75712	2.63895	1.88812	**1.72507**	18.2255	4.21775	3.98853	2.74420	3.59994	3.67531	3.63100	3.19791	2.94343	1.97304	**1.91575**
MRMSE	0.07021	0.01734	0.01482	0.01085	0.01422	0.01480	0.01462	0.01297	0.01257	0.00878	**0.00809**	0.11807	0.02714	0.02570	0.01766	0.02323	0.02375	0.02338	0.020637	0.019028	0.01260	**0.01221**
Time (sec)	-	145.504	**35.460**	111.224	83.921	39.761	42.986	167.6242	161.926	128.870	216.205	-	104.811	**20.370**	69.604	49.212	22.638	31.598	112.103	100.674	91.576	112.021
Mixture Noise																						
MPSNR	22.8028	34.4631	34.5747	36.1343	34.1641	33.9329	35.6393	35.9381	35.6571	36.4855	**37.7220**	17.5642	24.4057	28.6558	29.5417	29.7671	28.6209	29.8675	30.3806	29.5019	31.1162	**31.5603**
MSSIM	0.65107	0.95297	0.96194	0.97233	0.96704	0.96249	0.9721	0.97403	0.97166	0.97588	**0.97874**	0.32064	0.83033	0.81642	0.85218	0.87599	0.83189	0.87269	0.88163	0.87413	0.91315	**0.92259**
MFSIM	0.83781	0.97592	0.97759	0.98270	0.97776	0.97653	0.98296	0.98327	0.9819	0.98488	**0.98743**	0.68832	0.92407	0.92280	0.93157	0.93757	0.92524	0.94242	0.94386	0.93452	0.95876	**0.96242**
MERGAS	17.7275	4.96331	4.52498	3.75777	4.82799	4.92975	3.89761	3.76822	4.00748	3.73142	**3.23996**	22.5420	10.8906	6.60417	5.84054	5.94134	6.72551	5.75775	5.35732	6.02267	4.87293	**4.49944**
MRMSE	0.08459	0.02436	0.02176	0.01817	0.02291	0.02345	0.018424	0.017892	0.019299	0.01828	**0.01599**	0.14632	0.07066	0.04293	0.03789	0.03865	0.04379	0.037327	0.034792	0.03913	0.03164	**0.02916**
Time (sec)	-	121.141	**34.799**	110.497	82.562	39.725	39.358	167.6351	160.7322	128.778	214.935	-	269.273	**20.165**	69.154	49.331	22.608	29.568	111.548	100.436	92.244	111.359

F. Performance Comparison and Discussion

In all the simulated experiments, the parameters of the compared methods were manually adjusted to the optimal. The number of Gaussian components in the mixture of Gaussian noise is $k = 1$ in cases 1 and 2, and $k = 3$ in cases 3−6 for the NMoG-LRMF method. In addition, the stop criterion is $t = 10^{-4}$. For LRMR, the patch size is $20 \times 20 \times B$, and the step size is 8. In all the experiments of the LRTV method, we set the TV and the sparsity regularization parameters to $\tau = 0.015$ and $\lambda = 100/(M \times N)^{1/2}$, where M and N are the number of rows and columns in each band, respectively. For the CLV-LRMF, the 3-D Gaussian filter parameters are $\sigma = 0.1$ and $S = 25$ in the Washington DCMall data set, and $\sigma = 10$ and $S = 51$ in the RemoteImage data set. The rank of all low-rank-based methods is 5. The parameters of the LRTDTV method were manually adjusted and $C = 200$ is selected in $\lambda = 100 \times C/(M \times N)$ and rank is $r = [M, N, 5]$, and in the LLRSSTV, parameters λ and τ were manually fixed as 0.25 and 0.01; the patch size of each local patch was set as 20×20.

All parameters of different denoising strategies were carefully chosen by the related literature to obtain the best evaluation indexes and guarantee their possibly good performance. All methods are repeated ten times, with the initialization random parameter, and the average result is reported.

In Table II, the performance of the nine compared methods and the proposed CLV-LRMF method (over different noise settings for six types of noises) is listed. The results of MPSNR, MSSIM, MFSIM, MERGAS, MRMSE, and TIME for the Washington DCMall and RemoteImage as synthetic data sets are shown in this table. We can see that the proposed method achieves the best results with clear improvement on each data set and all investigated noises in the first five

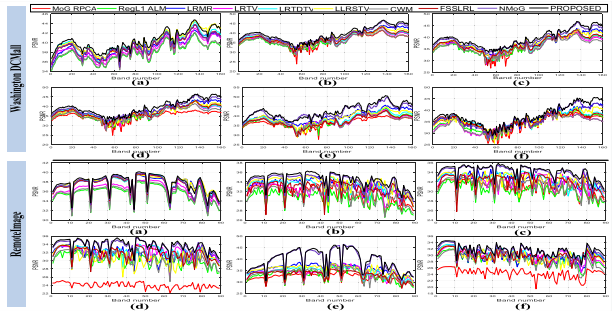


Fig. 6. MPSNR results of the Washington DCMall and RemoteImage data. (a) I.I.D. Gaussian. (b) Non-i.i.d. Gaussian. (c) Stripe-added non-i.i.d. Gaussian. (d) Deadline-added non-i.i.d. Gaussian. (e) Impulse-added non-i.i.d. Gaussian. (f) Mixture noise for MoG-RPCA, RegL1-ALM, LRMR, LRTV, CWM, FSSLRL, LRTDTV, LLRSTV, NMoG-LRMF, and proposed CLV-LRMF.

criteria with respect to the other compared methods, which are MoG-RPCA, RegL1-ALM, LRMR, LRTV, CWM, FSSLRL, LRTDTV, LLRSSTV, and NMoG-LRMF. According to these, the proposed method needs more time than others for running over both data sets.

The PSNR values across all bands are displayed under six types of noises in Fig. 6 for the Washington DCMall and RemoteImage data sets, respectively. The PSNR criterion behavior varies for both data sets and also for every band; however for both, the curve of the proposed CLV-LRMF method in most of the bands is higher than other curves of the compared methods. The robustness of our proposed CLV-LRMF algorithm over the entire HSI bands is shown by these results.

In Figs. 7 and 8, the SSIM and FSIM values are plotted across all bands for different noises and Washington DCMall and RemoteImage, respectively. For the SSIM and FSIM values close to 1, the two images (restored and noisy images)

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:06:47 UTC from IEEE Xplore. Restrictions apply.

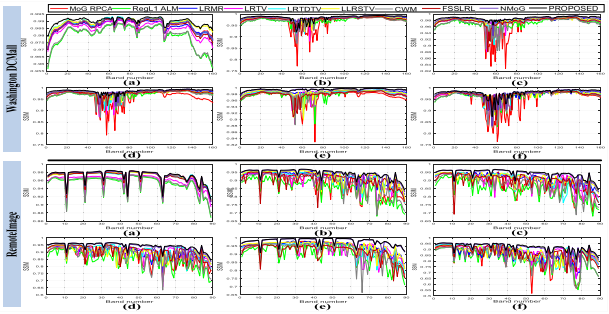


Fig. 7. MSSIM results for the Washington DCMall and RemoteImage data. (a) I.I.D. Gaussian. (b) Non-i.i.d. Gaussian. (c) Stripe-added non-i.i.d. Gaussian. (d) Deadline-added non-i.i.d. Gaussian. (e) Impulse-added non-i.i.d. Gaussian. (f) Mixture noise for MoG-RPCA, RegL1-ALM, LRMR, LRTV, CWM, FSSLRL, LRTDTV, LLRSTV, NMoG-LRMF, and proposed CLV-LRMF.

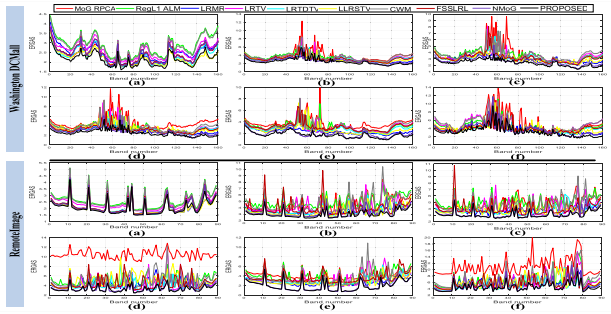


Fig. 9. MERGAS results of the Washington DCMall and RemoteImage data. (a) I.I.D. Gaussian. (b) Non-i.i.d. Gaussian. (c) Stripe-added non-i.i.d. Gaussian. (d) Deadline-added non-i.i.d. Gaussian. (e) Impulse-added non-i.i.d. Gaussian. (f) Mixture noise for MoG-RPCA, RegL1-ALM, LRMR, LRTV, CWM, FSSLRL, LRTDTV, LLRSTV, NMoG-LRMF, and proposed CLV-LRMF.

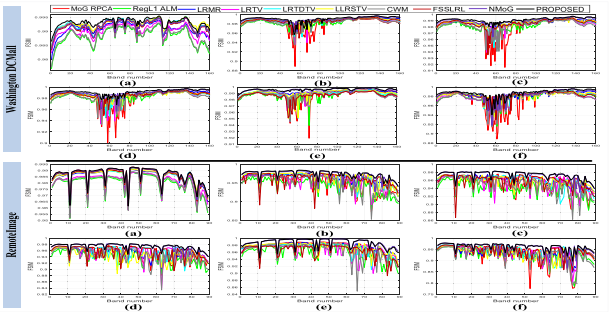


Fig. 8. MFSIM results in the Washington DCMall and RemoteImage data. (a) I.I.D. Gaussian. (b) Non-i.i.d. Gaussian. (c) Stripe-added non-i.i.d. Gaussian. (d) Deadline-added non-i.i.d. Gaussian. (e) Impulse-added non-i.i.d. Gaussian. (f) Mixture noise for MoG-RPCA, RegL1-ALM, LRMR, LRTV, CWM, FSSLRL, LRTDTV, LLRSTV, NMoG-LRMF, and proposed CLV-LRMF.

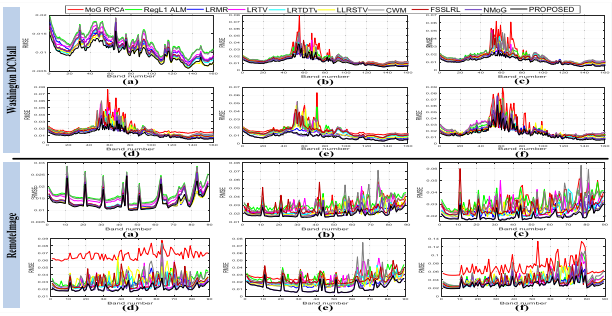


Fig. 10. MRMSE results of the Washington DCMall and RemoteImage data. (a) I.I.D. Gaussian. (b) Non-i.i.d. Gaussian. (c) Stripe-added non-i.i.d. Gaussian. (d) Deadline-added non-i.i.d. Gaussian. (e) Impulse-added non-i.i.d. Gaussian. (f) Mixture noise for MoG-RPCA, RegL1-ALM, LRMR, LRTV, CWM, FSSLRL, LRTDTV, LLRSTV, NMoG-LRMF, and proposed CLV-LRMF.

are similar and structurally equivalent. We can see that both SSIM and FSIM curves for the CLV-LRMF method stand above the others and are closer to one, which demonstrates the convergence of our method.

In the following simulations results, Figs. 9 and 10 show the ERGAS and RMSE values of each band with different noises for the Washington DCMall and RemoteImage data sets. The ERGAS and RMSE values in the CLV-LRMF method are smaller than the other six methods in both experimental data sets, indicating that the CLV-LRMF outperforms the other methods in the HSI noise removal.

For a better comparison of the visual details, we draw a 3-D view of all pixel values for one band. The color spectrum displayed in each image represents the pixels' values. In Fig 11, the 3-D views are obtained for the 53rd band of the original, noisy (that is damaged by the mixture noise), and restored versions (using MoG-RPCA, RegL1-ALM, LRMR, LRTV, CWM, FSSLRL, LRTDTV, LLRSTV, NMoG-LRMF, and our proposed CLV-LRMF algorithms) of the RemoteImage data set.

According to these, the spectrum restored by CLV-LRMF is much closer to the clean HSI, which in most details of the images are properly restored. Therefore, it performs best among all the compared methods and effectively removes the noise, preserving the essential structures of the original HSIs.

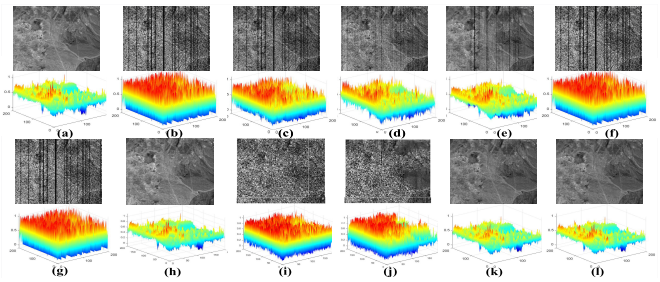


Fig. 11. Restoration results of band 53 of the RemoteImage data sets under mixture noise. Images are displayed in two ways: gray level and 3-D view. (a) Original HSI. (b) Noisy HSI. (c) MoG-RPCA. (d) RegL1-ALM. (e) LRMR. (f) LRTV. (g) CWM. (h) FSSLRL. (i) LRTDTV. (j) LLRSTV. (k) NMoG-LRMF. (l) Proposed CLV-LRMF.

In general, the proposed approach achieves more promising performance.

Each pixel of the HSI exhibits an approximately continuous spectrum across all bands, which is called the spectral profile. We illustrated the spectral profiles of the pixel (85 115) from the Washington DCMall data in the clean, noisy, and restored versions in Fig. 12. In these figures, the horizontal axis denotes the number of spectral bands and the vertical axis indicates the spectrum values. The recovered results by the CLV-LRMF method seem to be much closer to the original ones.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:06:47 UTC from IEEE Xplore. Restrictions apply.

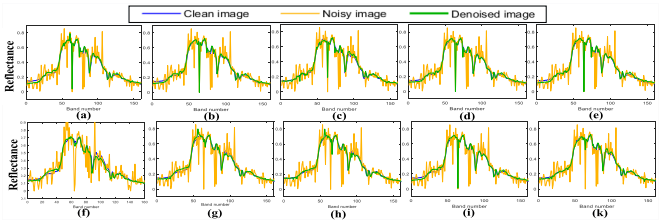


Fig. 12. Spectral characteristic for the pixels located at the position 85 × 115 of the Washington DCMall data set animated by mixture noise. Three curves corresponding to clean, noisy, and denoised images. (a) MoG-RPCA. (b) RegL1-ALM. (c) LRMR. (d) LRTV. (e) CWM. (f) FSSLRL. (g) LRTDTV. (h) LLRSTV. (i) NMoG-LRMF. (j) Proposed CLV-LRMF.

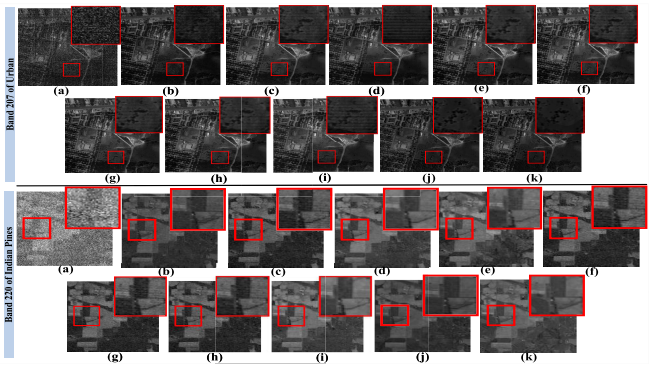


Fig. 13. Restoration results of all methods on bands 207 and 220 of the Urban and Indian Pines data sets. (a) Original noisy image. (b) MoG-RPCA. (c) RegL1-ALM. (d) LRMR. (e) LRTV. (f) CWM. (g) FSSLRL. (h) LRTDTV. (i) LLRSTV. (j) NMoG-LRMF. (k) Proposed CLV-LRMF method.

Fig. 13 shows bands 207 and 220 of the Urban and Indian Pines data sets, its restored results, and the corresponding enlarged images for one local, respectively. These are corrupted primarily by stripes. From a visual interpretation of the results of Urban, the proposed CLV-LRMF and NMoG-LRMF methods both have good performances. We can see that MoG-RPCA and LRMR fail to restore the image details and most of them are lost. The RegL1-ALM, LRTV, CWM, FSSLRL, LRTDTV, and LLRSTV can just remove an insignificant portion of noise, and the details in the scene are blurred and oversmoothed.

Due to the results of Indian Pines, RegL1-ALM, CWM, and LRTDTV are weaker than others in removing the noise in this real-world data set. LRMR, LRTV, FSSLRL, and LLRSTV can acquire relatively better results. The CLV-LRMF method can preserve the details and reduce the noise, and so predict the missing pixels.

In Fig. 14, we show the horizontal mean profiles of band 207 for the Urban data set and the vertical mean profiles for band 220 of the Indian Pines data set before and after restoration. As shown in Fig. 14, due to the existence of mixed noise, there are rapid fluctuations in the horizontal and vertical mean profile curves. We can evaluate the performance of algorithms by the smoothness of the mean profiles; therefore, MoG-RPCA and LRMR produce the worst results. RegL1-ALM, CWM, LRTV, FSSLRL, and LRTDTV achieve better performance. NMoG-LRMF and CLV-LRMF generate the best (and similar) results. The improved performance of CLV-LRMF is due to the latent variable step, because it can extract more noise information from the noisy data set.

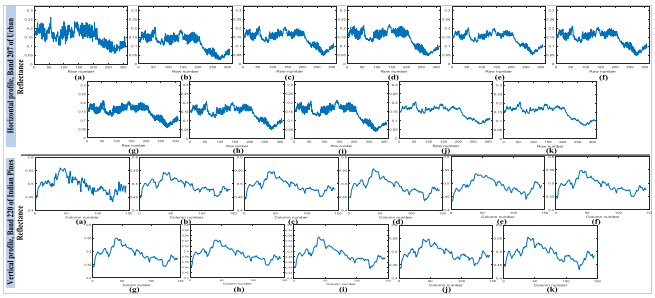


Fig. 14. Horizontal profile of band 207 in the Urban data set, and vertical profile of band 220 in the Indian Pines data set. (a) Original HSI. (b) MoG-RPCA. (c) RegL1-ALM. (d) LRMR. (e) LRTV. (f) CWM. (g) FSSLRL. (h) LRTDTV. (i) LLRSTV. (j) NMoG-LRMF. (k) Proposed CLV-LRMF method.

TABLE III

"NO-REFERENCE HSI QUALITY ASSESSMENT VIA QUALITY-SENSITIVE FEATURE LEARNING [44]" RESULTS FOR THE PROPOSED METHOD (CLV-LRMF) AND OTHER STATE-OF-THE-ART METHODS

Data set	Quality Score								
	MoG RPCA	RegL1-ALM	LRMR	LRTV	CWM	FSSLRL	LRTDTV	LLRSTV	NMoG-LRMF	CLV-LRMF
Urban	11.4755	11.6321	11.6453	11.7515	11.6200	11.8166	11.6261	11.6571	11.5623	**11.3604**
Indian Pines	16.1472	15.9111	16.3323	15.6403	15.8870	16.2636	15.8174	16.5302	16.9584	**14.3613**

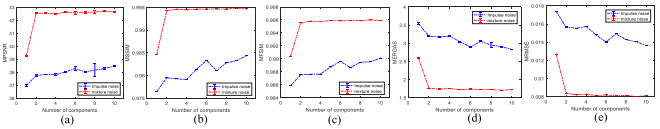


Fig. 15. Stability test of the proposed method to the Gaussian component number k on the Washington DCMall data for impulse and mixture noise. Sustainability of the criterion behavior has been investigated with respect to k for (a) MPSNR, (b) MSSIM, (c) MFSIM, (d) MRMSE, and (e) MERGAS.

The quality-assessment criteria such as PSNR, SSIM, FSIM, ERGAS, and RMSE requires a clean reference image to be available, which often is not in reality. In this article, we used a quality sensitive feature extraction-based method [44] for evaluating the quality of the real HSI data sets, because they have no reference image. In order to assess the quality of the reconstructed HSI, we must divide it into different local parts and use a multivariate Gaussian (MVG) model to fit on each part. We used the quality-sensitive features that are extracted in [44] for the AVIRIS images.

In addition, due to the robustness of MVG over the training data, we used the quality-sensitive features of the AVIRIS data in the quality assessment of the HYDICE data set. The score of the reconstructed HSI with a lower value represents higher quality. According to Table III, the CLV-LRMF and MoG-RPCA methods have the best results with the lowest values 11.3604 and 11.4755 on the Urban data, respectively. Similar to the Indian Pines data, CLV-LRMF and LRTV have the best results with the lowest values 14.3613 and 15.6403, respectively. Finally, we investigated the consequence of increasing parameter k as the number of Gaussian components in the mixture noise. These results have been achieved for the Washington DCMall data set and five evaluation criteria with k varying from 1 to 10. From Fig. 15, increasing the number of Gaussian components to two leads to performance improvement of CLV-LRMF. After $k \geq 3$, the performance

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:06:47 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
CLV-LRMF ALGORITHM TIME IMPROVEMENT VIA GPU

Noise Type	i.i.d. Gaussian	Non i.i.d. Gaussian	Gaussian +Stripe	Gaussian + Deadlines	Gaussian + Impulse	Mixture Noise	Average Time
Washington DCMall Data Timing (Sec)							
CLV Step on CPU	200.362	169.942	191.055	210.957	216.205	214.935	200.576
CLV Step on GPU	120.901	115.934	150.559	150.221	150.457	149.920	139.665
Time Deference	79.461	54.008	40.496	60.736	65.748	65.015	60.919
Improvement	39.65%	31.78%	21.19%	28.79%	30.41%	30.24%	30.34%
RemoteImage Data Timing (Sec)							
CLV step on CPU	105.387	93.854	114.019	115.807	112.021	111.359	108.741
CLV Step on GPU	80.799	79.680	95.479	97.190	95.908	95.630	90.781
Time Deference	24.588	14.174	18.540	18.617	16.113	15.729	17.960
Improvement	23.33%	15.10%	16.26%	16.07%	14.38%	14.12%	16.54%

will be stabilized and is not very sensitive to the choice of k value.

G. CLV-LRMF Time Improvement Using GPU

According to Table II, the CLV-LRMF results were better than the other methods in all criteria except time. This is because the proposed step (CLV) that is an extraction operator based on clustering takes more time to extract the CLVs. For example, the average time of applying the FCM clustering method on the DCMall data set with i.i.d. Gaussian noise was about 83 s. However, for the same data with the same noise, the average time needed for the whole CLV-LRMF method is 200.362 s. To manage the runtime and solve this challenge, our solution is using GPU. Therefore, to this end, we coded the CLV step to run on the GPU. The results are calculated after ten runs and then averaged for the Washington DCMall and RemoteImage data sets with six types of noises. The results of the comparison between the implementation time of the proposed method with a CPU and GPU in six types of noises are shown in Table IV. The CPU timing was carried out on an Intel Core i7 4790K CPU at 4 GHz with 32 GB of RAM, and GPU timing was done on a GeForce GTX 760. As shown in the table, using the GPU, the time was reduced from 83 to 4 and from 200.362 to 120.901 s for running the CLV step and the entire program, for the Washington DCMall data set with i.i.d. Gaussian, respectively. Therefore, the implementation time of the CLV-LRMF method was improved by 39%. By comparing the time results obtained from Table IV with those of Table II, you see that by exploiting GPU, the runtime of the CLV-LRMF is close to the other methods.

IV. CONCLUSION

In this article, we used the probabilistic model for noise into the Bayesian framework. In addition, we proposed the new CLV-LRMF. Our method is able to reduce the non-i.i.d. mixture of Gaussian noise and delete the effect of five other kinds of noises (e.g., non-i.i.d. Gaussian, stripe, deadline, impulse, and mixture noise), simultaneously. By finding the CLV, the CLV-LRMF method used the beneficial information of similar areas to estimate the noise destruction level more adequately. Therefore, these homogeneous partitioned structures in HSI data cube promote evaluation criteria MPSNR, MSSIM, and so on in the denoising process. As you see in the simulation results, the proposed CLV leads to improvement in the overall performance of the CLV-LRMF method and better noise removal for all types of noises, which damages the HSI. Compared with the current state-of-the-art techniques, the CLV-LRMF method performed more robustly due to its capability of adapting to several noise patterns.

REFERENCES

[1] A. F. H. Goetz, "Three decades of hyperspectral remote sensing of the Earth: A personal view," *Remote Sens. Environ.*, vol. 113, pp. 5–16, Sep. 2009.

[2] R. M. Willett, M. F. Duarte, M. A. Davenport, and R. G. Baraniuk, "Sparsity and structure in hyperspectral imaging: Sensing, reconstruction, and target detection," *IEEE Signal Process. Mag.*, vol. 31, no. 1, pp. 116–126, Jan. 2014.

[3] Y. Chen, X. Cao, Q. Zhao, D. Meng, and Z. Xu, "Denoising hyperspectral image with non-i.i.d. Noise structure," *IEEE Trans. Cybern.*, vol. 48, no. 3, pp. 1054–1066, Mar. 2018.

[4] W. He, H. Zhang, L. Zhang, and H. Shen, "Total-variation-regularized low-rank matrix factorization for hyperspectral image restoration," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 1, pp. 178–188, Jan. 2016.

[5] S. Z. Selim and M. A. Ismail, "K-means-type algorithms: A generalized convergence theorem and characterization of local optimality," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-6, no. 1, pp. 81–87, Jan. 1984.

[6] N. R. Pal and J. C. Bezdek, "On cluster validity for the fuzzy c-means model," *IEEE Trans. Fuzzy Syst.*, vol. 3, no. 3, pp. 370–379, Aug. 1995.

[7] H. Zhang, W. He, L. Zhang, H. Shen, and Q. Yuan, "Hyperspectral image restoration using low-rank matrix recovery," *IEEE Trans. Geosci. Remote Sens.*, vol. 52, no. 8, pp. 4729–4743, Aug. 2014.

[8] J. Xue, Y. Zhao, W. Liao, and J. C.-W. Chan, "Nonlocal low-rank regularized tensor decomposition for hyperspectral image denoising," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 7, pp. 5174–5189, Jul. 2019.

[9] T. Lin and S. Bourennane, "Hyperspectral image processing by jointly filtering wavelet component tensor," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 6, pp. 3529–3541, Jun. 2013.

[10] B. Rasti, J. R. Sveinsson, M. O. Ulfarsson, and J. A. Benediktsson, "Hyperspectral image denoising using first order spectral roughness penalty in wavelet domain," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2458–2467, Jun. 2014.

[11] L.-F. Dong, Y.-Z. Gan, X.-L. Mao, Y.-B. Yang, and C. Shen, "Learning deep representations using convolutional auto-encoders with symmetric skip connections," in *Proc. NIPS*, Apr. 2018.

[12] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3142–3155, Jul. 2017.

[13] Q. Yuan, Q. Zhang, J. Li, H. Shen, and L. Zhang, "Hyperspectral image denoising employing a spatial–spectral deep residual convolutional neural network," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 1205–1218, Feb. 2019.

[14] Y.-Q. Zhao and J. Yang, "Hyperspectral image denoising via sparse representation and low-rank constraint," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 1, pp. 296–308, Jan. 2015.

[15] W. He, H. Zhang, H. Shen, and L. Zhang, "Hyperspectral image denoising using local low-rank matrix recovery and global spatial–spectral total variation," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 3, pp. 713–729, Mar. 2018.

[16] Y. Wang, J. Peng, Q. Zhao, Y. Leung, X.-L. Zhao, and D. Meng, "Hyperspectral image restoration via total variation regularized low-rank tensor decomposition," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 4, pp. 1227–1243, Apr. 2018.

[17] L. Sun, B. Jeon, B. N. Soomro, Y. Zheng, Z. Wu, and L. Xiao, "Fast superpixel based subspace low rank learning method for hyperspectral denoising," *IEEE Access*, vol. 6, pp. 12031–12043, 2018.

[18] K. Dabov, A. Foi, V. Katkovnik, and K. Egiazarian, "Image denoising by sparse 3-D transform-domain collaborative filtering," *IEEE Trans. Image Process.*, vol. 16, no. 8, pp. 2080–2095, Aug. 2007.

[19] M. Elad and M. Aharon, "Image denoising via sparse and redundant representations over learned dictionaries," *IEEE Trans. Image Process.*, vol. 15, no. 12, pp. 3736–3745, Dec. 2006.

[20] L. Liu, L. Chen, C. L. P. Chen, Y. Y. Tang, and C. M. Pun, "Weighted joint sparse representation for removing mixed noise in image," *IEEE Trans. Cybern.*, vol. 47, no. 3, pp. 600–611, Mar. 2017.

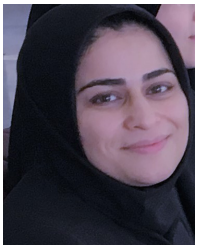
[21] L. Shao, R. Yan, X. Li, and Y. Liu, "From heuristic optimization to dictionary learning: A review and comprehensive comparison of image denoising algorithms," *IEEE Trans. Cybern.*, vol. 44, no. 7, pp. 1001–1013, Jul. 2014.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:06:47 UTC from IEEE Xplore. Restrictions apply.

[22] P. Zhong and R. Wang, "Multiple-spectral-band CRFs for denoising junk bands of hyperspectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 4, pp. 2260–2275, Apr. 2013.
[23] D. Letexier and S. Bourennane, "Noise removal from hyperspectral images by multidimensional filtering," *IEEE Trans. Geosci. Remote Sens.*, vol. 46, no. 7, pp. 2061–2069, Jul. 2008.
[24] X. Liu, S. Bourennane, and C. Fossati, "Denoising of hyperspectral images using the PARAFAC model and statistical performance analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 10, pp. 3717–3724, Oct. 2012.
[25] K. Dabov, A. Foi, and K. Egiazarian, "Video denoising by sparse 3D transform-domain collaborative filtering," in *Proc. Eur. Signal Process. Conf.*, Poznan, Poland, Sep. 2007, pp. 145–149.
[26] M. Maggioni, V. Katkovnik, K. Egiazarian, and A. Foi, "Nonlocal transform-domain filter for volumetric data denoising and reconstruction," *IEEE Trans. Image Process.*, vol. 22, no. 1, pp. 119–133, Jan. 2013.
[27] H. Fan, C. Li, Y. Guo, G. Kuang, and J. Ma, "Spatial–spectral total variation regularized low-rank tensor decomposition for hyperspectral image denoising," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 10, pp. 6196–6213, Oct. 2018.
[28] X. Bai, F. Xu, L. Zhou, Y. Xing, L. Bai, and J. Zhou, "Nonlocal similarity based nonnegative tucker decomposition for hyperspectral image denoising," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 3, pp. 701–712, Mar. 2018.
[29] G. Ely, S. Aeron, and E. L. Miller, "Exploiting structural complexity for robust and rapid hyperspectral imaging," 2013, *arXiv:1305.2170*. [Online]. Available: http://arxiv.org/abs/1305.2170
[30] B. Rasti, J. R. Sveinsson, and M. O. Ulfarsson, "Wavelet-based sparse reduced-rank regression for hyperspectral image restoration," *IEEE Trans. Geosci. Remote Sens.*, vol. 52, no. 10, pp. 6688–6698, Oct. 2014.
[31] T. Hu, H. Zhang, H. Shen, and L. Zhang, "Robust registration by rank minimization for multiangle hyper/multispectral remotely sensed imagery," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2443–2457, Jun. 2014.
[32] Q. Li, H. Li, Z. Lu, Q. Lu, and W. Li, "Denoising of hyperspectral images employing two-phase matrix decomposition," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 9, pp. 3742–3754, Sep. 2014.
[33] W. He, H. Zhang, L. Zhang, and H. Shen, "A noise-adjusted iterative randomized singular value decomposition method for hyperspectral image denoising," in *Proc. IEEE Geosci. Remote Sens. Symp. (IGARSS)*, Quebec City, QC, Canada, Jul. 2014, pp. 1536–1539.
[34] Q. Zhao, D. Meng, Z. Xu, W. Zuo, and L. Zhang, "Robust principal component analysis with complex noise," in *Proc. 31st Int. Conf. Mach. Learn.*, Beijing, China, 2014, pp. 55–63.
[35] Y. Zheng, G. Liu, S. Sugimoto, S. Yan, and M. Okutomi, "Practical low-rank matrix approximation under robust L1-norm," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Providence, RI, USA, Jun. 2012, pp. 1410–1417.
[36] D. Meng, Z. Xu, L. Zhang, and J. Zhao, "A cyclic weighted median method for L1 low-rank matrix factorization with missing entries," in *Proc. Assoc. Adv. Artif. Intell.*, Bellevue, WA, USA, vol. 4, 2013, p. 6.
[37] L. Fang, S. Li, X. Kang, and J. A. Benediktsson, "Spectral–spatial hyperspectral image classification via multiscale adaptive sparse representation," *IEEE Trans. Geosci. Remote Sens.*, vol. 52, no. 12, pp. 7738–7749, Dec. 2014.
[38] L. Fang, S. Li, X. Kang, and J. A. Benediktsson, "Spectral–spatial classification of hyperspectral images with a superpixel-based discriminative sparse model," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 8, pp. 4186–4201, Aug. 2015.
[39] L. Fang, S. Li, W. Duan, J. Ren, and J. A. Benediktsson, "Classification of hyperspectral images by exploiting spectral–spatial information of superpixel via multiple kernels," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 12, pp. 6663–6674, Dec. 2015.
[40] Y. Yuan, J. Lin, and Q. Wang, "Dual-clustering-based hyperspectral band selection by contextual analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 3, pp. 1431–1445, Mar. 2016.
[41] Q. Wang, J. Lin, and Y. Yuan, "Salient band selection for hyperspectral image classification via manifold ranking," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 27, no. 6, pp. 1279–1289, Jun. 2016.
[42] L. Zhang, W. Wei, Y. Zhang, C. Shen, D. H. Van, and Q. Shi, "Cluster sparsity field: An internal hyperspectral imagery prior for reconstruction," *Int. J. Comput. Vis.*, vol. 126, pp. 797–821, Mar. 2018.
[43] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
[44] J. Yang, Y. Zhao, C. Yi, and J. C.-W. Chan, "No-reference hyperspectral image quality assessment via quality-sensitive features learning," *Remote Sens.*, vol. 9, no. 4, p. 305, 2017.
[45] J. Liu, P. Musialski, P. Wonka, and J. Ye, "Tensor completion for estimating missing values in visual data," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 1, pp. 208–220, Jan. 2013.
[46] H. Song, G. Wang, and K. Zhang, "Hyperspectral image denoising via low-rank matrix recovery," *Remote Sens. Lett.*, vol. 5, no. 10, pp. 872–881, Oct. 2014.
[47] F. Fana, Y. Maa, C. Lib, X. Meia, J. Huanga, and J. Maa, "Hyperspectral image denoising with superpixel segmentation and low-rank representation," *Inf. Sci.*, vols. 397–398, pp. 48–68, Feb. 2017.
[48] C. Zou and Y. Xia, "Poissonian hyperspectral image superresolution using alternating direction optimization," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 9, no. 9, pp. 4464–4479, Sep. 2016.



Peyman Azimpour received the B.S. degree in geomatics engineering from Tehran University, Tehran, Iran, in 2017. He is pursuing the M.S. degree in remote sensing engineering with the Ferdowsi University of Mashhad, Mashhad, Iran. ' His research interests include hyperspectral image analysis, spectral unmixing, signal processing, and pattern recognition in remote sensing imagery.



Tahereh Bahraini received the B.S. degree in electrical engineering from the University of Persian Gulf (PGU), Bushehr, Iran, in 2011, and the M.S. degree in electrical engineering from the Shiraz University of Technology (SUTech), Shiraz, Iran, in 2013, respectively. She is pursuing the Ph.D. degree in electrical engineering with the Ferdowsi University of Mashhad (FUM), Mashhad, Iran.

Her research interests include pattern recognition, signal processing, statistical modeling, and optimization.



Hadi Sadoghi Yazdi received the B.S. degree in electrical engineering from the Ferdowsi University of Mashhad (FUM), Mashhad, Iran, in 1994, and the M.S. and Ph.D. degrees in electrical engineering from Tarbiat Modares University, Tehran, Iran, in 1996 and 2005, respectively.

He is a Professor of computer science and engineering with FUM. His research interests are in the areas of pattern recognition, machine learning, machine vision, signal processing, and optimization.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 02:06:47 UTC from IEEE Xplore. Restrictions apply.